TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

KỊCH BẢN THUYẾT TRÌNH BẢO VỆ ĐỒ ÁN CUỐI KỲ
HỆ THỐNG QUẢN LÝ HOẠT ĐỘNG HỘI NHẬP – VISIONARY

Mô hình Một Web API (ASP.NET Core 8) – Ba Nền tảng (Next.js, Windows Forms, Android)
Thời lượng chuẩn: Đúng 15 phút trình bày | Khai báo AI: Google Antigravity hỗ trợ kiểm chứng mã nguồn

Thứ tự	Thành viên	Phần nội dung phụ trách & Slide	Thời lượng mục tiêu
1	Quách Khoa Đức (Trưởng nhóm)	Slide 1–4: Mở đầu, Bài toán thực tế, Kiến trúc hệ thống & Web Client	2 phút 45 giây (Dễ nhớ nhất)
2	Đào Trí Khang	Slide 5–7: Ứng dụng Android Native, Luồng mạng/UI & Demo Android	3 phút 15 giây
3	Hồ Đoàn Nguyên Khang	Slide 8–11: Swagger Contract, 6 chặng request, JWT 401/403 & DB	4 phút 15 giây
4	Võ Ngọc Minh Đạt	Slide 12–16: WinForms Desktop, Demo điểm danh, 3 khác biệt, Sự cố & Kết	4 phút 45 giây (Kết thúc bài)

PHẦN 1: QUÁCH KHOA ĐỨC (SLIDE 1 – 4 | 2 PHÚT 45 GIÂY)

Slide 1 – Trang bìa & Giới thiệu nhóm (45 giây)

Lời thoại:

"Kính chào Thầy và các bạn trong lớp! Em tên là Quách Khoa Đức, trưởng nhóm VISIONARY API.
Nhóm em gồm 4 thành viên: em, bạn Đào Trí Khang, bạn Hồ Đoàn Nguyên Khang và bạn Võ Ngọc Minh Đạt.
Hôm nay, nhóm em rất vinh dự được đại diện báo cáo sản phẩm đồ án môn học Web API (TH71108) với đề tài: VISIONARY – Hệ thống Quản lý Hoạt động Hội nhập dành cho Câu lạc bộ Tiếng Anh và sinh viên DNTU.
Hệ sinh thái của nhóm được thiết kế chuẩn mực theo mô hình một Web API duy nhất viết bằng ASP.NET Core 8 kết nối cơ sở dữ liệu PostgreSQL trên nền tảng đám mây, phục vụ đồng bộ cho ba ứng dụng: Website Next.js, Desktop Windows Forms và Mobile Android Native.
Về việc sử dụng AI, nhóm em khai báo minh bạch có sử dụng Google Antigravity để hỗ trợ rà soát mã nguồn, thiết lập kiểm thử hồi quy tự động và biên soạn tài liệu; toàn bộ mã nguồn, cấu hình đám mây và kết quả chạy thực tế đều do nhóm em trực tiếp kiểm chứng và vận hành."

Slide 2 – Bài toán thực tế & Mục tiêu hệ thống (45 giây)

Lời thoại:

"Thưa Thầy, lý do nhóm em chọn đề tài này xuất phát từ bài toán rất thực tế tại trường Đại học Công nghệ Đồng Nai.

Trước đây, các hoạt động của CLB Tiếng Anh và việc kết nối sinh viên tham gia các cuộc thi hội nhập chủ yếu quản lý qua Google Form và file tính rời rạc, dẫn đến dữ liệu dễ thất lạc và Ban Chủ Nhiệm khó kiểm soát số lượng điểm danh.

Sinh viên CNTT cũng thiếu một cổng thông tin tập trung về học liệu English for IT và các kho công nghệ mở thế giới.

Mục tiêu của VISIONARY là số hóa toàn diện hoạt động của CLB: sinh viên dùng điện thoại Android để đăng ký sự kiện và nhận mã QR; Ban Chủ Nhiệm dùng phần mềm Windows Forms để điểm danh và quản lý; còn website Next.js là cổng tin tức công khai cho toàn trường."

Slide 3 – Kiến trúc hệ thống: Một API, Ba ứng dụng (45 giây)

Lời thoại:

"Trên màn hình là sơ đồ kiến trúc hệ thống của nhóm em.

Ở trung tâm là máy chủ Web API viết bằng ASP.NET Core 8 chạy trên web server Kestrel, lắng nghe trên cổng 443 HTTPS trên nền tảng Render cloud, hoặc cổng 5000 khi chạy máy cá nhân.

Cả ba ứng dụng đều giao tiếp với API qua giao thức HTTP chuẩn RESTful.

Website Next.js chạy trên trình duyệt gọi vào API qua cổng 443.

Ứng dụng Windows Forms chạy trên hệ điều hành Windows gọi trực tiếp qua HttpClient.

Riêng ứng dụng Android, khi chạy trên máy ảo sẽ gọi qua địa chỉ đặc biệt 10.0.2.2 vì máy ảo coi localhost là chính nó; còn khi chạy thực tế trên điện thoại, Android gọi thẳng vào tên miền cloud của server.

Phía sau API là cơ sở dữ liệu PostgreSQL trên đám mây Supabase Singapore kết nối qua cổng 5432.

Việc dùng chung một API giúp hệ thống đồng bộ dữ liệu tức thời và kiểm soát phân quyền an toàn tại một nơi duy nhất."

Slide 4 – Ứng dụng Web Client (Next.js 14) & Chuyển lời (30 giây)

Lời thoại:

"Đây là giao diện Web của nhóm được xây dựng bằng Next.js 14 App Router, chạy tại địa chỉ visionary-platform.vercel.app.

Các điều khiển trên màn hình tuân thủ đúng quy ước đặt tên: ô tìm kiếm là txtSearch, nút bấm là btnSearch, danh sách sự kiện là grdEvents.

Đoạn mã bên phải lấy trực tiếp từ file events/page.tsx: khi người dùng gõ từ khóa, hàm gửi yêu cầu GET tới /api/events kèm phân trang.

Nhóm đã áp dụng AbortController để tự động hủy request cũ khi người dùng gõ nhanh, chống lỗi bất đồng bộ race condition.

Sau đây, em xin kính mời bạn Đào Trí Khang tiếp tục trình bày về ứng dụng di động Android Native."

PHẦN 2: ĐÀO TRÍ KHANG (SLIDE 5 – 7 | 3 PHÚT 15 GIÂY)

Slide 5 – Ứng dụng Di động Android Native Java (1 phút)

Lời thoại:

"Kính chào Thầy và các bạn! Em tên là Đào Trí Khang, phụ trách phát triển ứng dụng di động Android Native bằng ngôn ngữ Java.

Ứng dụng Visionary Connect được thiết kế tối ưu cho sinh viên DNTU theo dõi các hoạt động CLB và đăng ký sự kiện ngay trên điện thoại.

Bên trái là ảnh chụp thật màn hình ứng dụng trên máy Android.

Các điều khiển giao diện tuân thủ quy ước: danh sách cuộn là lstEvents (RecyclerView), nút đăng ký là btnRegister, vuốt làm mới là swpRefresh.
Đoạn mã bên phải nằm trong EventsFragment.java: hàm getEvents() gọi tới Web API thông qua thư viện Retrofit 2.
Yêu cầu được thực thi trên luồng nền bằng hàm enqueue(), khi có kết quả onResponse sẽ tự động đưa về luồng chính để cập nhật adapter hiển thị."

Slide 6 – Android gọi API: Luồng nền & Luồng giao diện (1 phút)

Lời thoại:

"Thưa Thầy, một trong những thách thức cốt lõi khi lập trình Android kết nối Web API là quản lý đa luồng.
Hệ điều hành Android có hai quy tắc nghiêm ngặt: Thứ nhất, cấm tuyệt đối gọi mạng trên Luồng Chính (UI Thread) để tránh đơ máy; nếu gọi đồng bộ, Android sẽ ném ngoại lệ NetworkOnMainThreadException và tắt app ngay.
Thứ hai, cấm tuyệt đối cập nhật giao diện View từ luồng nền; nếu vi phạm sẽ gặp lỗi CalledFromWrongThreadException.
Nhóm em đã giải quyết bằng phương thức enqueue() của Retrofit: thư viện OkHttp sẽ mở luồng nền thực hiện kết nối mạng và parse JSON.
Sau đó, Retrofit sử dụng Handler gắn với Looper.getMainLooper() để đưa kết quả quay trở lại Luồng Chính cập nhật RecyclerView một cách an toàn."

Slide 7 – Demo trực tiếp ứng dụng Android (1 phút 15 giây)

Lời thoại:

"Bây giờ em xin thực hiện demo trực tiếp trên bản build APK thật của ứng dụng Android.
Đầu tiên, em đăng nhập bằng tài khoản sinh viên daotrikhang@visionary.dntu.edu.vn. Khi đăng nhập thành công, app nhận token JWT và lưu vào bộ nhớ SharedPreferences.
Tại tab sự kiện, app gọi GET /api/events hiển thị danh sách các workshop sắp diễn ra.
Em chọn sự kiện 'Master Git, GitHub & CI/CD Actions' và bấm nút 'ĐĂNG KÝ NGAY'.
Ứng dụng gửi yêu cầu POST /api/events/{id}/register kèm mã JWT token.
Máy chủ kiểm tra sức chứa và hạn đăng ký, sau đó lưu bản ghi vào PostgreSQL và trả về mã QR check-in VIS-QR-E4B2A8C1.
Màn hình ngay lập tức chuyển sang trạng thái ĐÃ ĐĂNG KÝ.
Dữ liệu này đã được đồng bộ ngay lập tức lên hệ thống. Tiếp theo, em xin kính mời bạn Hồ Đoàn Nguyên Khang báo cáo phần Web API và Swagger."

PHẦN 3: HỒ ĐOÀN NGUYÊN KHANG (SLIDE 8 – 11 | 4 PHÚT 15 GIÂY)

Slide 8 – Swagger UI & Hợp đồng API (1 phút)

Lời thoại:

"Kính chào Thầy và các bạn! Em tên là Hồ Đoàn Nguyên Khang, phụ trách máy chủ Web API, tài liệu Swagger và cơ sở dữ liệu.
Bên trái là giao diện Swagger UI đang chạy trực tuyến tại địa chỉ https://visionary-api-gb51.onrender.com/swagger.
Bảng bên phải là hợp đồng API chính thức của nhóm, khớp chính xác từng ký tự với phản hồi của server.
Hợp đồng phân chia rõ ràng các cấp độ quyền:
Endpoint đăng nhập và xem danh sách sự kiện là công khai, ai cũng gọi được.
Endpoint đăng ký sự kiện yêu cầu người dùng phải đăng nhập có Bearer token.
Endpoint tạo sự kiện và điểm danh sinh viên bắt buộc phải có vai trò Admin hoặc Ban Chủ Nhiệm (Committee).
Toàn bộ các trường dữ liệu đều dùng chuẩn camelCase như totalCount, pageIndex, pageSize và items."

Slide 9 – Xuyên suốt một yêu cầu: 6 chặng dữ liệu (1 phút)

Lời thoại:

"Để chứng minh sự liên kết giữa ba nền tảng, nhóm em chọn endpoint GET /api/events và POST /api/events/register đi qua đủ 6 chặng:

Chặng 1: Bắt đầu từ cú nhấp chuột của người dùng trên nút giao diện.

Chặng 2: Hàm UI gọi lớp ApiClient tương ứng của từng nền tảng: Web dùng fetch, WinForms dùng HttpClient.SendAsync, Android dùng Retrofit enqueue.

Chặng 3: Gói tin HTTP mang header Authorization Bearer gửi qua mạng tới máy chủ Kestrel cổng 443.

Chặng 4: Server tiếp nhận, middleware xác thực chữ ký JWT, controller gọi Entity Framework Core truy vấn cơ sở dữ liệu PostgreSQL.

Chặng 5: Server bọc dữ liệu vào ApiResponse<T>, serialize thành chuỗi JSON UTF-8 và trả về mã HTTP 200 OK.

Chặng 6: Client nhận JSON, deserialize thành model DTO, chuyển về Luồng Chính và đổ lên bảng dữ liệu hoặc thẻ giao diện."

Slide 10 – Xác thực & Phân quyền JWT Bearer (1 phút 15 giây)

Lời thoại:

"Về cơ chế xác thực, nhóm em sử dụng JSON Web Token theo chuẩn RFC 7519.

Hai hình ảnh trên slide là kết quả kiểm thử thực tế của nhóm:

Hình bên trái là mã HTTP 401 Unauthorized khi gọi endpoint quản trị mà không truyền Bearer token. Server từ chối ngay tại middleware UseAuthentication(), không để lộ dữ liệu.

Hình bên phải là mã HTTP 403 Forbidden: trường hợp này người dùng đã đăng nhập thành công với vai trò Sinh viên (Student), nhưng cố tình gọi vào API cập nhật quyền của Admin thì server từ chối ngay bằng thuộc tính [Authorize(Roles = 'Admin')].

Điểm quan trọng nhất: JWT không phải là chuỗi mã hóa bí mật, mà là chuỗi được ký số. Bất kỳ ai cũng đọc được phần thân, nhưng không ai có thể sửa được vai trò vì sửa là chữ ký số HMAC-SHA256 sẽ sai ngay lập tức.

Ngoài ra, Server luôn tự trích xuất UserId từ Claims trong token chứ không tin vào dữ liệu người dùng gửi lên."

Slide 11 – JSON, Cơ sở dữ liệu & Quyết định kỹ thuật (1 phút)

Lời thoại:

"Ở tầng kiến trúc dữ liệu, nhóm em đưa ra ba quyết định quan trọng:

Thứ nhất, luôn dùng DTO (Data Transfer Object) để tách biệt giữa bảng cơ sở dữ liệu và dữ liệu trả ra ngoài. DTO giúp che giấu mật khẩu đã băm PasswordHash và triệt tiêu lỗi vòng lặp tuần tự hóa JSON.

Thứ hai, cơ chế phân trang PagedResponse bắt buộc thực thi ở cấp cơ sở dữ liệu bằng Skip và Take, kết hợp tiêu chí sắp xếp phụ tie-breaker theo Id để đảm bảo không bị trùng lặp bài giữa các trang.

Thứ ba, cơ sở dữ liệu PostgreSQL đặt tại Singapore kết nối qua IPv4 Connection Pooler giúp container backend kết nối ổn định.

Sau đây, em xin kính mời bạn Võ Ngọc Minh Đạt trình bày ứng dụng Windows Forms, sự cố thật và tổng kết bài báo cáo."

PHẦN 4: VÕ NGỌC MINH ĐẠT (SLIDE 12 – 16 | 4 PHÚT 45 GIÂY)

Slide 12 – Ứng dụng Desktop Windows Forms C# (1 phút)

Lời thoại:

"Kính chào Thầy và các bạn! Em tên là Võ Ngọc Minh Đạt, phụ trách ứng dụng Windows Forms C# trên .NET 8.

Đây là phần mềm chuyên dụng dành cho Ban Chủ Nhiệm CLB để quản trị danh sách thành viên, duyệt đơn và điểm danh sự kiện trên máy tính văn phòng.

Các điều khiển sử dụng chuẩn: lưới sự kiện là _dgvEvents, lưới người đăng ký là _dgvRegistrations, nút điểm danh là btnCheckIn.

Đoạn mã bên phải nằm trong MainForm.cs: nhóm sử dụng từ khóa async và await kết hợp với đối tượng HttpClient tĩnh duy nhất.
Từ khóa await giúp nhường quyền điều khiển cho giao diện trong lúc chờ máy chủ phản hồi, sau đó tự động quay về đúng luồng giao diện khi có kết quả.
Nhờ vậy, cửa sổ ứng dụng luôn mượt mà và tuyệt đối tránh được lỗi Deadlock do gọi .Result hay .Wait()."

Slide 13 – Demo trực tiếp ứng dụng Windows Forms (1 phút 15 giây)

Lời thoại:

"Bây giờ em xin mở bản build Release thực tế của ứng dụng Windows Forms để demo nghiệp vụ điểm danh.
Em đăng nhập bằng tài khoản Ban Chủ Nhiệm vongocminhdat@visionary.dntu.edu.vn.
Tại tab 'Sự kiện & Điểm danh', khi em nhấp vào workshop Git, bảng đăng ký phía dưới lập tức hiển thị thông tin bạn Đào Trí Khang vừa bấm đăng ký từ điện thoại Android ở Slide 7.
Em chọn dòng sinh viên này và bấm nút 'Xác nhận Điểm danh (Check-in)'.
Ứng dụng gửi yêu cầu POST tới /api/events/{id}/attendance.
Máy chủ kiểm tra quyền BCN của em, ghi nhận thời gian điểm danh và trả về thành công.
Cột trạng thái trên màn hình lập tức đổi sang 'Đã điểm danh'.
Đồng thời, khi mở Website hoặc Android xem lại, trạng thái đã được cập nhật đồng bộ 100% mà không bị lệch dữ liệu."

Slide 14 – Ba chỗ buộc phải khác nhau giữa ba nền tảng (1 phút)

Lời thoại:

"Thưa Thầy, một câu hỏi quan trọng là: Điều gì buộc phải giống nhau và điều gì buộc phải khác nhau giữa ba ứng dụng?
Nhóm em đã tổng hợp rõ trong bảng so sánh:
Ba chỗ buộc phải khác nhau do đặc thù ngôn ngữ và hệ điều hành:
1. Thư viện mạng: JavaScript dùng fetch(), C# dùng HttpClient, còn Java Android dùng Retrofit và OkHttp.
2. Điều phối luồng giao diện: Android bắt buộc dùng Handler thủ công, C# có async/await tự động quay về UI context, còn trình duyệt chạy trên Event Loop.
3. Nơi giữ token: Web lưu trong localStorage, WinForms lưu trên biến RAM tĩnh, Android lưu trong SharedPreferences mã hóa.
Ngược lại, điều bắt buộc phải giống nhau tuyệt đối là Hợp đồng API: cùng đường dẫn, cùng phương thức HTTP, cùng khuôn JSON và mã trạng thái.
Đây chính là minh chứng rõ nhất cho giá trị của kiến trúc tách tầng Web API."

Slide 15 – Một sự cố thật và Quy trình Khắc phục (1 phút)

Lời thoại:

"Trong quá trình kiểm thử tự động, nhóm em đã đối mặt với một sự cố kỹ thuật có thật trên GitHub Actions Backend CI Run #12.
Bên trái là log lỗi nguyên văn: pipeline báo Build FAILED dù 0 lỗi biên dịch C#, với 13 bài đạt và 2 bài kiểm thử thất bại (bài Search dòng 168 và bài GetPostBySlug dòng 183).
Đi sâu vào log, nhóm phát hiện thông báo lỗi SQLite Error 19: UNIQUE constraint failed: Roles.Name.
Nguyên nhân gốc rễ do xUnit kích hoạt hai class kiểm thử chạy song song, cùng khởi tạo WebApplicationFactory trỏ vào chung một file visionary.db, gây xung đột ghi đồng thời khiến chu trình nạp dữ liệu SeedAsync bị dừng đột ngột trước khi kịp nạp bài viết.
Nhóm em đã tự phân tích và xử lý triệt để bằng giải pháp hai tầng: Thứ nhất, đặt khóa luồng tĩnh SemaphoreSlim(1, 1) trong DbInitializer; Thứ hai, xây dựng CustomWebApplicationFactory cấp phát cơ sở dữ liệu test cô lập và gộp chung Collection kiểm thử.
Như hình bên phải, toàn bộ 15/15 bài test đã vượt qua 100% và GitHub Actions Run #40231783994 trên Linux đã báo xanh hoàn toàn."

Slide 16 – Giới hạn thực tế, Hướng phát triển & Kết thúc (30 giây)

Lời thoại:

"Kính thưa Thầy, biết mình chưa làm được gì là một dạng hiểu biết chứ không phải lời thú tội.
Dù hệ thống đã chạy tốt, nhóm em thẳng thắn nhìn nhận ba hạn chế thực tế:
Thứ nhất, chưa có thông báo đẩy thời gian thực qua SignalR hoặc Firebase.
Thứ hai, ứng dụng Android chưa có bộ nhớ đệm ngoại tuyến Room Database khi mất mạng.
Thứ ba, cơ chế tự động cấp lại token (Refresh Token) cần được hoàn thiện hơn.
Nếu có thêm hai tuần, nhóm em sẽ ưu tiên tích hợp SignalR và Room DB để nâng cao độ bền vững của ứng dụng.
Đến đây, nhóm VISIONARY API xin kết thúc bài thuyết trình.
Chúng em xin chân thành cảm ơn Thầy và các bạn đã chú ý lắng nghe! Nhóm chúng em sẵn sàng bước vào phần phản biện ạ!"